

The Operative Note Intelligence Engine: Architecture, Evidence, and What Must Be Built

A discussion document for NeuroFlo

Companion to: *Operative Note Requirements & CPT Mapping — Cardiovascular and General Surgery* (v1.0) Prepared: September 2026

Preface: what this document is, and why it is written the way it is

The first document I produced for you was a research artifact. It was organized by *source of authority* — what the Centers for Medicare & Medicaid Services requires, what the National Correct Coding Initiative prohibits, what Health Level Seven's document standard specifies, what the American College of Surgeons mandates for cancer programs. That organization is correct for the purpose of establishing what is true and where the truth comes from. It is the wrong organization for the purpose of instructing an artificial intelligence system at the moment a surgeon presses "analyze."

This document is different. It is written as an argument rather than a catalog. Its purpose is to establish, before anything is built, four things: what the actual problem is and whether the evidence supports the way you are currently describing it; what the correct architecture for the knowledge base is, and why the obvious architecture is wrong; what specific artifacts must exist for your multi-agent system to function; and what information you must obtain before those artifacts can be built honestly.

I have read your strategic breakdown in full — the three-module framing, the return-on-investment mathematics, the chief financial officer pitch slide, the competitive analysis, and the Blue Ocean narrative. I agree with the strategic instinct almost entirely. I disagree with several of the specific numbers, and I want to explain why, because the disagreement matters: the numbers you have chosen are the ones a sophisticated hospital finance executive will attack first, and there are better numbers available that are both more defensible and, in the places that matter most, more persuasive.

Throughout, I have tried to define every term of art at the point of first use, because the vocabulary in this domain is dense and much of it is used loosely even by people inside the industry.

Part One: The central architectural argument

1.1 The thesis

You cannot validate an operative note against a Current Procedural Terminology code. You can only validate it against the facts that select the code.

This is the single most important structural decision in the entire system, and getting it wrong will produce an engine that appears to work in demonstration and fails in production. Let me build the argument carefully.

Current Procedural Terminology — universally abbreviated CPT — is the code set maintained by the American Medical Association that describes procedures and services for billing purposes. When people say "the surgery was coded," they mean a trained human read the operative note and selected one or more CPT codes that describe what was done. A CPT code is therefore not a property of the surgery. It is a *conclusion* drawn from a description of the surgery.

This distinction sounds academic. It is not. Consider what happens if you build the knowledge base as a collection of CPT codes with associated documentation requirements — the intuitive design, and the one most teams reach for. Your agent receives a note. It must now do something like: guess the code, look up the code's requirements, check whether the note satisfies them. But the guess is the hard part, and the guess is unreliable, and every downstream check inherits the unreliability of the guess. Worse, when the agent guesses wrong, it will confidently check the note against the wrong requirements and report that a complete note is incomplete, or that an incomplete note is fine. That failure mode is invisible to the user and corrosive to trust.

Now consider the inverse design. The knowledge base is a collection of **facts** — discrete, atomic, checkable assertions that may or may not appear in a note. "Total hernia defect length, measured before opening the defect." "Number of arterial coronary grafts." "Whether the lesion was a stenosis or an occlusion." "Whether the appendix was diseased." The agent's first job is not to guess a code. Its first job is a far easier and far more reliable one: *read the note and determine, for each relevant fact, whether it is stated, absent, or ambiguous.*

Codes then fall out of the facts deterministically. If total defect length is under three centimeters, and the hernia is initial rather than recurrent, and it is reducible rather than incarcerated, then the code is 49591. That is not a judgment call. It is a lookup on a three-dimensional grid. The American College of Surgeons describes the selection procedure in exactly these terms: (cite index="101-1">First, determine the total length (size); then choose the type of hernia (initial versus recurrent); and finally, choose the severity (reducible or incarcerated/strangulated).</cite>

The architectural consequence is that you have split an unreliable single-step task into a reliable two-step pipeline. The first step — fact extraction from prose — is precisely what large language models are good at. The second step — mapping facts to codes — is deterministic table logic that a language model should never touch, because it is exactly the kind of reasoning where models are least reliable and where errors are least detectable.

There is a further consequence, and it is the one that matters most for your product. **A missing fact is actionable; a wrong code is not.** If your system tells a surgeon "you coded 49593 but I think it should be 49591," you have started an argument the surgeon has no reason to lose, and you have positioned the software as a billing enforcement tool. If your system tells the surgeon "you wrote *approximately three centimeters* — the code depends on whether that is above or below three, and this measurement cannot be reconstructed after the fact," you have given a clinician a reason to act that is grounded in their own documentation, not in your opinion about money. The second framing is the one that gets adopted. It also happens to be the accurate one.

1.2 The taxonomy of facts

If facts are the atomic unit, then facts need a classification, because different classes of fact carry different consequences when missing, and the system must respond differently to each. I propose four classes.

Legal-mandatory facts. These are required by federal regulation, specifically 42 CFR §482.51(b)(6) and its interpretive guidance, and by whichever accrediting body the hospital has chosen. They must be present in every operative note regardless of procedure. Their absence makes the note non-compliant as a document, independent of any billing consideration. Examples: the times of surgery, the type of anesthesia, an affirmative statement about complications. A hospital cannot configure these away, and your system should decline to let them try.

Code-selecting facts. These determine which CPT code is correct. Their absence does not make the note illegal, but it makes the note *uncodeable at the correct level* — the coder must default downward to the least specific applicable code, and the difference is real money that cannot be recovered later. These are the facts where the timing of your intervention is everything, because most of them are unrecoverable once the patient leaves the operating room.

Modifier-supporting facts. A modifier is a two-character suffix appended to a CPT code that alters its meaning — indicating, for instance, that the procedure was unusually complex, or performed bilaterally, or performed by two surgeons of different specialties working simultaneously. Modifiers frequently carry additional payment, and payers audit them aggressively. Each modifier has a specific assertion that must exist in the note narrative to support it. Absent that assertion, the modifier is unsupported and the claim is exposed.

Registry and quality facts. These are required by voluntary programs the hospital participates in — the Society of Thoracic Surgeons Adult Cardiac Surgery Database, the Society for Vascular Surgery's Vascular Quality Initiative, the American College of Surgeons Commission on Cancer. Their absence costs nothing in billing terms but creates abstraction gaps that a registry coordinator must later chase down through manual chart review, and in the cancer-program case can genuinely threaten accreditation.

The reason this taxonomy matters operationally is **severity assignment**. If your system treats all four classes with equal urgency, it will produce twenty alerts on a routine case and surgeons will stop reading them. If it distinguishes them, it can present two blocking items, three warnings, and quietly log the rest. That difference determines whether the product is used or bypassed, and it is a knowledge-base design decision, not a user-interface decision.

1.3 Why detection must come first, and the question I cannot answer without you

Everything above presumes the system knows what operation it is looking at. A hernia profile applied to a coronary bypass note produces nonsense. So the first thing the pipeline must do is classify the note: what specialty, what procedure family, what specific sub-procedure, what anatomical site, what laterality, what approach.

You identified this yourself and then set it aside as something to handle later. I would argue it must come first, because every downstream artifact is gated on it. But before I can build the detection criteria, I need to know something about your data flow that I do not currently know.

Does the surgical case posting reach your system?

In hospital operating room workflow, every case is booked in advance with a scheduled procedure description, a scheduled surgeon, a service line, a laterality, and often a scheduled CPT code. This data lives in the surgical scheduling module and, in most Cerner and Epic environments, is exposed through the standard interoperability interfaces as a scheduled procedure or encounter resource. If that data reaches you, detection is largely solved before the note is even written. You know it is a coronary artery bypass because someone booked it as one. The detection artifact then becomes a modest disambiguation layer handling the cases where the booked procedure and the performed procedure diverge — which happens, and is itself worth flagging, because a divergence between booked and performed procedure is a strong signal that something notable occurred.

If, on the other hand, you receive only note text, detection becomes a substantial classification problem in its own right, requiring a hierarchical taxonomy, a large synonym and eponym dictionary, and confidence thresholds with a fallback to asking the surgeon directly. It is solvable, but it is a different and much larger artifact.

This is the single biggest fork in the road for the work ahead, and I do not want to build the wrong thing. **I need you to tell me which situation you are in.**

Part Two: Validating the problem, and correcting the numbers

You asked me to help validate the problem and supply defensible validation points. I want to do that honestly, which means separating the claims in your strategy document that hold up from the ones that do not. The good news is that the underlying problem is real, well-documented, and quantifiable. The bad news is that several of the specific figures you are using are either wrong, unsourced, or attributed to bodies that did not publish them — and a hospital chief financial officer or a healthcare-focused investor will find those quickly.

I will take them in order of how much damage they would do if challenged.

2.1 The documentation time claim needs revision

Your document states that surgeons spend thirty to forty-five minutes per operative note, and fifteen to twenty percent of their workday on documentation. I could not find support for the per-note figure, and the best available surgeon-specific measurement points the other way.

A study using electronic health record audit logs across surgical subspecialties at an academic center found that (cite index="171-1">surgeons spent 57.5 minutes per day logged into the EHR, 19.1 minutes per day in the EHR outside the scheduled hours, and 19.4 minutes per day in the EHR outside the scheduled days</cite>. That is total electronic health record time — notes, orders, results review, messages, everything. A surgeon performing three cases in a day cannot be spending thirty to forty-five minutes per operative note within a 57.5-minute total. The claim is arithmetically impossible against the best measurement available.

This does not mean documentation burden is not real. It is extremely well established. A narrative review of twenty-five studies of electronic health record burnout in surgery found that (cite index="175-1">documentation [burden appeared in] 15/25 = 60.0% (95% CI = 40.7-76.6); time demands, 17/25 = 68.0% (95% CI = 48.4-82.8)</cite> of the included studies, concluding that (cite index="176-1">In U.S. surgical literature, EHR-related burnout is most frequently reported in association with documentation burden and time demands.</cite>

So the burden is real and the burnout linkage is real. The per-note number is not defensible. My recommendation is to stop quantifying per-note minutes entirely and instead lead with the after-hours figure, which is both accurate and emotionally sharper: **surgeons are spending roughly twenty minutes per day in the electronic record outside scheduled hours, and roughly another twenty on days they are not scheduled to work.** That is documentation displacing personal time, measured by system audit logs rather than self-report. It lands harder than a fabricated per-note figure, and it survives scrutiny.

2.2 The denial rework cost is wrong, and the denial rate is understated

Your document cites \$118 as the average cost to rework a denied claim, and a ten to fifteen percent denial rate. Both need updating.

On cost, Premier's analysis puts the figure substantially lower: (cite index="159-1">Hospitals and health systems spent an estimated \$25.7 billion adjudicating claims with payers in 2023, up 23% from 2022, with the cost per denied claim rising to \$57.23</cite>. Using \$118 when the most-cited industry source says \$57.23 is the kind of error that causes a chief financial officer to stop trusting the rest of your model.

On rate, the current data is actually *better* for your argument than what you cited. Kodiak Solutions, drawing on one of the largest revenue cycle datasets in the country, reports that (cite index="153-1">The average initial denial rate also increased to 11.6% in 2025, from 11.4% in 2024</cite>, based on (cite index="153-1">aggregated and normalized revenue cycle data from 2,300 hospitals and 350,000 physicians</cite>. But the headline number understates the problem for surgical inpatient work specifically: (cite index="154-1">It found that 21% of inpatient claims in the commercial population were initially denied, although the final denial rate was only 3%.</cite>

Twenty-one percent of commercial inpatient claims initially denied. That is your population — complex surgical inpatient cases — and it is roughly double the all-payer average.

The composition shift matters even more than the rate. Kodiak's characterization of the trend is directly on point for what you are building: (cite index="156-1">Payors are paying cleaner, simpler claims faster, but the harder work is getting harder—with clinical initial denials continuing to rise and providers increasingly being asked to 'prove' medical necessity, which is driving higher final denials.</cite> And separately: (cite index="159-1">medical necessity denials rose while prior authorization denials fell. Payers are moving the fight downstream, contesting whether the care was justified and documented rather than only whether it was authorized.</cite>

Read that carefully, because it is the strongest market-timing argument available to you and it is not in your current deck. **Payers have shifted the battleground from authorization to documentation.** The fight is no longer about whether the procedure was approved; it is about whether the note proves it was necessary and proves what was done. A company whose entire product is making the note prove those things is positioned exactly where the market pressure is moving. That is a "why now" argument grounded in third-party benchmarking data across 2,300 hospitals, and it is far more compelling than a generic assertion that denials are bad.

The aggregate figure is also worth having: (cite index="155-1">Kodiak Solutions reported that net revenue leakage among the hospitals in its dataset increased 25% from 2024 to 2025, rising from \$38.6 billion to \$48.4 billion.</cite>

One caution, in the interest of your credibility rather than your enthusiasm. Premier also reports that (cite index="159-1">about 70% of denials are ultimately overturned</cite>. A sharp finance executive will use this against you — if most denials are eventually paid, the loss is administrative rather than absolute. The correct response is to concede the point and reframe: the cost of a denial that is eventually overturned is the rework expense plus the cash-flow delay plus the staff capacity consumed, and Premier's own framing of that waste is precise — (cite index="159-1">Every dollar spent re-litigating a correct claim is a dollar that produced no new care and no new revenue.</cite> Conceding the overturn rate and pivoting to waste is more persuasive than pretending the overturn rate does not exist, and it signals that you understand their business.

2.3 The DNFB benchmark does not exist, and this is a serious exposure

This is the correction I would most urge you to make before the next investor or hospital conversation.

DNFB stands for "discharged, not final billed" — the population of patient accounts where the patient has left the hospital but no bill has been generated, usually because coding or documentation is incomplete. It is measured in days: total unbilled discharged dollars divided by average daily revenue. It is a genuine and important metric, and hospital finance leaders watch it daily.

Your document states a "7.1 day industry median" and attributes the metric to the Healthcare Financial Management Association, generally abbreviated HFMA, which is the professional body for hospital finance. The attribution is half right and the number appears

to be invented.

HFMA does define the metric. It is one of their MAP Keys, which is their published set of standardized revenue cycle performance indicators: (cite index="164-1">The days in total discharged not final billed is one the of the Healthcare Financial Management Association's (HFMA) MAP Keys. This key indicates the revenue cycle's performance and pinpoints issues impacting cash flow for the organization.</cite>

But HFMA does not publish a target. This is stated explicitly by practitioners in the field: (cite index="167-1">You will often hear people cite a "3 to 5 day" DNFB figure. Treat that as a common operational rule of thumb that many revenue cycle teams manage toward, not as an official benchmark published by a standards body. HFMA defines how to calculate the metric, but it does not publish a public numeric target you are required to hit. The right target for your organization depends on your case mix, your payer mix, and your own baseline.</cite>

Where numbers do circulate, they cluster around three to five days as a working target, with (cite index="163-1">Poorly performing hospitals have DNFB of 11.6 days or more</cite> and (cite index="168-1">For those with automated workflows and interoperable systems, 5-7 days is typical</cite>.

The problem for your model is compounding. You assume a seven-day baseline, assume a reduction to one day, multiply six days by five million dollars per day to produce thirty million dollars of freed working capital, and then headline that number. Every step is contestable. The baseline is not an established benchmark. A one-day DNFB is not achievable by any operative-note intervention, because DNFB is driven by many causes your product does not touch — (cite index="165-1">Incomplete Clinical Documentation... Coding Errors Or Backlogs... Denials And Rework... Workflow Inefficiencies</cite> — of which you address roughly one and a half. And the five-million-dollars-per-day figure appears to be gross charges rather than net revenue, which in most hospitals differ by a factor of two to four.

A chief financial officer will identify all three problems inside a minute, because DNFB is a number they personally own.

Here is what I would do instead, and I think it is genuinely more powerful. **Do not claim a DNFB number. Ask for theirs.** Walk in and say: you know your DNFB days, we do not, and we are not going to guess. Here is what we do know — a measurable share of your DNFB is physician documentation queries on surgical cases, meaning a coder read a note, could not code it, and sent a question back to a surgeon who is now in another operating room. Show us that share and we will tell you what fraction of it our engine would have prevented at the point of authoring.

That converts an attackable assertion into a discovery question the customer answers with their own data. It positions you as someone who understands the metric rather than someone reciting it. And the resulting number, whatever it is, is unassailable, because they produced it.

There is a real precedent for the effect, incidentally: (cite index="163-1">one company helped a client reduce DNFB by 56%. This released over \$25 million in cash for the client.</cite> Cite that as a directional analogue rather than a promise.

2.4 The charge leakage figure needs a source or a change of framing

You assume one to three percent revenue leakage from missed billable items. This is a widely repeated figure in revenue cycle marketing material, and I was not able to trace it to a primary study. It may well be accurate. But "widely repeated in vendor marketing" and "defensible under questioning" are different standards, and an investor who has heard the same one-to-three-percent figure from four other companies will discount it.

Fortunately you do not need it, because you have something better: a specific, mechanically verifiable, dollar-denominated example.

For anterior abdominal hernia repair, the code depends on the total defect length, and the measurement must be taken before the defect is opened. If the surgeon writes "approximately three centimeters," the coder is stuck. The trade press documents both the rule and the arithmetic: (cite index="98-1">if the op report states that the reducible hernia is approximately 3 cm, which is right on the cusp between codes, you will likely need to play it safe and report the code for a hernia that's less than 3 cm, which is 49591 for an initial repair rather than 49593 if the surgeon had accurately reported 3.5 cm. The payment for those two codes is \$345.65 and \$579.13 respectively (Medicare Physician Fee Schedule National payment amount, conversion factor 33.8872). That means missing accurate documentation would cost the surgeon \$233.48.</cite>

And critically, on recoverability: (cite index="98-1">Even if you were to query the provider, there's no way they can measure the pre-surgery hernia size retroactively. The same rings true if the provider did not report a measurement at all. You'll have to report the lowest code possible for the type of hernia.</cite>

That is two hundred thirty-three dollars and forty-eight cents, on a single common general surgery case, lost permanently, from one imprecise adjective. It is preventable by a prompt at the moment of writing and by nothing else afterward. It is not an estimate, not an industry average, and not a vendor statistic — it is arithmetic on the published Medicare fee schedule.

One verifiable case beats a percentage every time. I would build the entire leakage argument on this example and then say: this is one code family in one specialty, and we have mapped hundreds of these. That claim is checkable, which is exactly why it lands.

2.5 What the evidence genuinely and strongly supports

Having spent several pages on corrections, let me be equally clear about what holds up, because the core of your thesis is well supported and in one respect the literature makes your argument better than you are making it yourself.

Operative notes are measurably incomplete, and the deficiency does not improve with experience. A review of the surgical education literature found that (cite index="182-1">Multiple studies demonstrated significant deficiencies in residents' ODs, with key information missing in up to 76% of cases. The completeness of OD did not consistently correlate with level of training.</cite>

That second sentence is the important one and I want to dwell on it. If completeness improved with training, incompleteness would be a competence problem, and the solution would be education. It does not improve with training. Therefore it is not a competence problem — it is a **structural** problem, arising from the fact that unstructured recall under time pressure and fatigue omits items regardless of how skilled the recaller is. Structural problems require structural solutions. Software is a structural solution. Education is not. This single finding is the strongest intellectual justification for your product's existence, and it is not currently in your deck.

Structured reporting demonstrably fixes it. Two independent systematic reviews, published in 2019, reached the same conclusion from partly different study sets. The first performed a meta-analysis and found that (cite index="125-1">synoptic reporting was significantly more complete (SMD 1.70, 95% CI 1.13 to 2.26; I2 98%). Completion time was shorter with synoptic reporting (mean difference -0.86, 95% CI -1.17 to -0.55). Secondary outcomes favoured synoptic reporting.</cite> A standardized mean difference of 1.70 is a very large effect. The second found that (cite index="124-1">Overall and subsection completion of the operative report was higher in the synoptic operative report. The time until completion of the operative report and the data extraction time were shorter in the synoptic report... The use of mandatory fields in the synoptic report resulted in more completely reported operative outcomes with completion percentages close to 100%.</cite>

Note that both found structured reporting to be **both more complete and faster**. That defeats the objection you will hear most often from surgeons, which is that a completeness tool will slow them down.

And here is the finding that should define your product architecture. The same review that found synoptic reporting superior overall identified precisely one domain where it lost: (cite index="124-1">One exception was the specific details section concerning the operative procedure, as this was generally reported more frequently in the narrative report.</cite> Its conclusion: (cite index="124-1">A hybrid approach to the synoptic operative report will potentially yield better completion rates and higher physician satisfaction.</cite>

Pure structure loses procedural nuance. Pure narrative loses completeness. The literature's own recommendation is a hybrid — narrative authoring with a structured completeness layer over it. That is exactly what you are building. You are not making an architectural bet; you are implementing the conclusion of the systematic reviews. That is a materially stronger position than "we think this is a good idea," and it is worth saying in exactly those terms.

Templates produce the gain regardless of who is writing. A general surgery trial of templated laparoscopic cholecystectomy notes found that (cite index="77-1">The completeness of data in template group was significantly better than traditional group (79.2% vs. 8.3%). There was no significant difference among the residents of different level (writing the notes) and the completeness of data in both the groups.</cite> Eight percent to seventy-nine percent, with the improvement independent of trainee seniority. Same conclusion as above, arrived at experimentally.

Similar magnitudes recur across audit studies: (cite index="182-1">In 2 studies, the use of synoptic report maximized the completion rate of OD up to 92% from less than 70%.</cite> And in a closed-loop audit of general surgical notes, (cite index="187-1">The overall level of compliance improved from about 80% at the initial audit to >95% at the re-audit.</cite>

An honest caveat you should carry knowingly. Much of the operative note audit literature comes from outside the United States — the United Kingdom, Pakistan, Sudan, Malawi — and is benchmarked against Royal College of Surgeons guidelines rather than CMS requirements. The Malawi study, for example, found baseline documentation of (cite index="180-1">time of performing the operation, urgency, estimated blood loss, complications and extra procedure in the first audit, with a frequency of 0%, 2%, 14%, 38% and 11% respectively</cite>. Those baselines are far worse than a modern American academic center would show, and citing them as though they represented United States practice would be misleading. Use this literature to establish that the *problem class* is universal and that *structured intervention resolves it*, which it does robustly. Do not use the absolute percentages as United States benchmarks. If someone challenges you on this, saying so yourself first is worth more than the citation.

2.6 A summary of the corrections

Before moving on, let me consolidate what I am recommending you change. The table below lists each contested claim in your current strategy document, what I would replace it with, and the reasoning.

Current claim	Recommended replacement	Why
30–45 min per operative note	Drop entirely; use ~19 min/day after-hours plus ~19 min/day on unscheduled days	Per-note figure is unsupported and arithmetically inconsistent with measured total EHR time
\$118 per denied claim rework	\$57.23 (Premier, 2023)	The widely cited industry figure; using a doubled number invites challenge
10–15% denial rate	11.6% all-payer initial (Kodiak 2025); 21% commercial inpatient	The inpatient number is both accurate and more favorable to you
7.1-day DNFB "HFMA median"	No claim; ask for the customer's own number	HFMA defines the metric but publishes no target; the median is unsourced
7 days → 1 day DNFB	No claim; scope to the physician-query component only	A note tool cannot address most DNFB drivers; overclaiming is easily disproved
\$30M liquidity unlocked	Derived from the customer's actual figures during discovery	Currently rests on three contestable assumptions multiplied together
1–3% charge leakage	The hernia case: \$233.48, one case, unrecoverable	Verifiable arithmetic beats an industry percentage
3,945% ROI	A payback framing on defensible components only	A four-digit ROI reads as unserious to finance professionals
— (not currently claimed)	Op note deficiency does not correlate with training level	Your strongest justification; establishes this as structural, not educational
— (not currently claimed)	Payers have shifted from authorization denials to documentation and medical-necessity denials	Your strongest "why now"; sourced to 2,300-hospital benchmarking

What that table shows is that the corrections are not subtractions. Five of the ten rows replace a weak claim with a stronger one, and two add arguments you were not making at all. The revised pitch is more conservative in its arithmetic and more aggressive in its positioning — which is the correct direction of travel when the audience is a hospital finance executive who has heard four vendor pitches this quarter.

Part Three: What must be built

I now want to describe the artifacts themselves — what each one is, what problem it solves, why it must be separate from the others,

and what it looks like in practice. There are six. I will argue for each in turn.

The reason there are six rather than one is worth stating up front. You asked for “a book” — a single reference the AI can consult. A single document is the wrong container for two reasons. First, different artifacts change on different schedules: the legal requirements change roughly never, the code tables change annually, the hospital's own preferences change continuously. Binding them together means every annual code update forces a review of the regulatory content, which is wasteful and error-prone. Second, different artifacts are consumed at different stages of the pipeline. The detection criteria are needed before anything else runs; the boilerplate library is needed after analysis is complete. A monolithic document forces every agent to load everything, which is slow, expensive, and dilutes attention.

Six artifacts, each with a single job, composed at runtime.

Artifact 1 — Detection Criteria

The job. Given whatever inputs are available — case posting, note text, or both — determine the specialty, the procedure family, the specific procedure, the anatomical site, the laterality, and the approach, with a confidence score. Route to the correct profile. Where confidence is insufficient, ask rather than guess.

Why it must exist separately. Every other artifact is selected by its output. A misdetection does not degrade the analysis; it invalidates it entirely. This artifact therefore needs its own accuracy target, its own test set, and its own failure behavior, none of which are shared with the downstream reasoning.

What it contains. A hierarchical procedure taxonomy for the three specialties in scope, running from specialty down through family down to individual procedure and variant. A large synonym dictionary, because surgeons do not write canonical procedure names — they write “CABG x3,” “triple,” “three-vessel,” “LIMA-LAD plus two SVG,” and all four denote the same thing. An eponym dictionary, because eponyms are pervasive in surgery and opaque to any system that has not been told what they mean: Whipple, Bentall, Cabrol, Ross, David, Nissen, Hartmann, Ivor Lewis, Fontan, Norwood, Glenn. Anatomical vocabulary with hierarchy, so that the system understands that the superficial femoral artery is within the femoral-popliteal territory, which is within the lower extremity vascular family. Approach detection. Laterality detection with explicit handling of bilateral and of unstated. Confidence thresholds and escalation rules.

The nuance that will bite you. Surgery is frequently multi-procedure. A single note may describe a coronary bypass, a mitral valve repair, and a left atrial appendage occlusion, performed in one session by one surgeon through one incision. The detector must return a set with a designated primary, not a single label, and the downstream profile composition must handle the union of their requirements while suppressing the duplicates. This is not an edge case; in cardiac surgery it is closer to the norm than the exception.

The one-off problem you raised. You are right that unusual and unprecedented operations occur, and any system claiming to enumerate all of them is lying. The correct handling is a graceful degradation ladder. If the specific procedure is recognized, apply the specific profile. If only the family is recognized, apply the family profile and inform the surgeon that specific checks were unavailable. If only the specialty is recognized, apply the specialty profile plus the universal requirements. If nothing is recognized, apply the universal requirements alone and say so plainly. The system should never silently apply a wrong profile, and it should never refuse to run. Every degradation should be visible to the user, because a surgeon who knows the system only checked the basics will not over-rely on it — and over-reliance on a tool that quietly failed is the most dangerous outcome available.

Artifact 2 — Element Dictionary

The job. Define, exactly once, every discrete fact that can appear in a surgical note. Establish the shared vocabulary that all other artifacts reference.

Why it must exist separately. Without it, every procedure profile independently reinvents “estimated blood loss,” and the definitions drift. Six months in you have four incompatible definitions of the same element, extraction logic duplicated in four places, and no way to answer the question “how often is estimated blood loss missing across all specialties” because the element is not a single thing in your data model. Defining elements once is the difference between a maintainable knowledge base and an unmaintainable one, and the cost of getting it wrong compounds silently.

What each entry contains. A stable identifier. A plain-language definition. A data type — is this a number with units, an enumerated choice, a free-text narrative, a Boolean, a date, a duration. Where enumerated, the permitted values. The classification from section 1.2 above: legal-mandatory, code-selecting, modifier-supporting, or registry. Whether the element permits an affirmative negative — that

is, whether "none" is a valid and required answer rather than simply an absence. The extraction surface forms, meaning the many ways surgeons actually write it. The typical section it lives in. Its default severity when missing. And the source authority establishing that it is required at all.

Why the surface forms matter more than anything else in this artifact. Consider estimated blood loss. In real notes it appears as "EBL," "E.B.L.," "Estimated blood loss," "Blood loss," "estimated blood loss was," "EBL:", and as bare values like "minimal," "scant," "nil," "<50 cc," "50 mL," "approximately 100 cc," "negligible." A system that matches on "estimated blood loss" and nothing else will report this element missing on the majority of real notes, generate false alerts, and be switched off within a week. False positives are more damaging than false negatives in clinical decision support, because a false negative is invisible while a false positive is an active annoyance that trains the user to dismiss without reading.

There is empirical support for how hard this is. A study analyzing section headers across 362,311 operative notes found that (cite index="76-1">Section headers contained large numbers of synonyms, formatting variation, and variation of word forms, as well as smaller numbers of compound sections and issues with mismatches in header granularity.</cite> It also found the encouraging part: (cite index="76-1">HL7-ON DSTU alone and HL7-ON DSTU with LOINC section headers covered 66% and 79% of sections headers (93% and 98% of header instances), respectively.</cite>

Read those two numbers together and the engineering strategy becomes clear. Seventy-nine percent of *distinct* headers covers ninety-eight percent of *header instances*. The long tail is enormous in variety and tiny in frequency. Build a comprehensive normalization map for the common forms — that gets you to ninety-eight percent deterministically, fast, cheaply, and auditably — and reserve language-model inference for the residual tail. Do not attempt to solve the whole problem with inference on every note. It is slower, more expensive, non-deterministic, and impossible to explain to a compliance officer.

And this is why I need real notes. The surface-form map cannot be written from first principles. I can generate a plausible list, but a plausible list is a list of the ways I imagine surgeons write, not the ways they actually write. Real dictated notes from real surgeons at two in the morning contain transcription artifacts, abbreviations, and constructions that no amount of reasoning will produce. De-identified operative notes from Vandalia — even a few hundred, even from one service — would change the quality of this artifact more than any other input you could give me.

Artifact 3 — Procedure Profiles

The job. For each procedure and sub-procedure, specify which elements from the dictionary are required, which are conditionally required and under what condition, and which are optional. This is the direct answer to your question "what is supposed to be in this operative note."

Why it must exist separately. This is the layer that varies by procedure, and it is the layer hospitals will want to extend. Keeping it separate from the element dictionary means a hospital can add a requirement to their cardiac profile without touching the shared definition of any element, and means you can ship a new procedure profile without regression-testing the entire knowledge base.

Structure. Each profile composes rather than duplicates. It inherits the universal requirements, then the specialty requirements, then adds procedure-specific ones. A coronary bypass profile does not restate that the note needs times and an anesthesia type; it inherits those. It adds cardiopulmonary bypass time, aortic cross-clamp time, cardioplegia strategy, conduit type per target vessel, number of distal anastomoses, prior sternotomy status.

Conditional logic is where the real content lives. Most requirements are not unconditional. They fire when something else is true. If an implant was used, then manufacturer, model, size, and anatomic position become required. If an assistant participated, then the specific tasks that assistant performed become required — and this is not a nicety but an explicit payer expectation: (cite index="113-1">The surgeon needs to specify in body of the operative report what tasks were performed by the assistant during the procedure. It is not sufficient evidence of participation to list the assistant's name in the [header].</cite> If the approach converted from laparoscopic to open, then the reason and timing of conversion become required, and the coding consequence changes entirely. If a biopsy was taken during a resection, then whether frozen section was performed and whether it altered the plan become required, because that single fact determines whether the biopsy is separately billable.

Flagging the unrecoverable. Within each profile, elements should be marked as to whether they can be captured after the fact. Most cannot. A defect measurement, a cross-clamp time, an intraoperative finding, a lesion characterization — none of these can be reconstructed once the case is closed. This flag is what drives the urgency of the prompt and what justifies interrupting a surgeon at all. An element that can be added tomorrow does not warrant an interruption today. An element that is gone forever does.

Artifact 4 — Code Selection Decision Trees

The job. For each CPT family, an ordered sequence of questions that resolves to a code, together with the add-on logic, the hierarchy logic, and the counting rules.

Why it must exist separately. This is the artifact that changes annually and sometimes catastrophically, and it is the artifact that must be deterministic. Isolating it means the annual American Medical Association update touches one file. It also means this logic can be tested exhaustively against known cases without involving a language model at all, which is essential for auditability.

What a tree looks like in practice. For anterior abdominal hernia: is this an anterior abdominal hernia, then what is the total defect length, then is this initial or recurrent, then is it reducible or incarcerated. Four questions, twelve terminal codes. Plus the special rules, which are where the errors actually happen: multiple defects are summed and reported once, not separately — (cite index="99-1">These codes [49591-49596, and 49613-49618] are reported only once, based on the total defect size, for however many abdominal hernias the patient has that are getting repaired in that surgical session</cite> — and mixed severity resolves upward, because (cite index="101-1">if there are multiple hernias (i.e., "Swiss cheese"), the entire repair defaults to the highest severity of any of the defects</cite>.

For coronary bypass the tree is different in shape but identical in principle. How many arterial grafts, how many venous grafts, and were both types used. If venous only, one code from the 33510 through 33516 range and nothing else. If arterial only, one code from 33533 through 33536. If both, an arterial base code plus a venous add-on. The governing rule is unambiguous: (cite index="57-1">When venous grafting only is performed, only one code in the range of coronary artery bypass CPT codes 33510-33516 may be reported. No other bypass codes shall be reported with these codes. One code in the range of CPT codes 33517-33523 (combined arterial-venous grafting) and one code in the range of CPT codes 33533-33536 (arterial grafting) may be reported together to accurately describe combined arterial-venous bypass.</cite>

The most common error in coronary bypass coding is reporting a venous-only code alongside an arterial code, which the rule above forbids explicitly. That is a single Boolean check and it is worth building on day one.

The urgent case: lower extremity revascularization. This deserves emphasis because it is time-sensitive and because the exposure at Vandalia is almost certainly live right now. Effective January 2026, (cite index="91-1">The existing 16 assigned codes (37220-37235) were deleted and replaced with 46 codes (37254-37299) that resulted in significant changes to documentation requirements.</cite> A fourth vascular territory was added. Reporting moved from territory-level to vessel-level, meaning the old convention that treated the entire femoral-popliteal territory as one unit no longer holds. And most consequentially for a documentation engine, lesion complexity became a code-selecting fact: (cite index="91-1">For lower extremity endovascular revascularization treatment, a straightforward lesion is a stenosis, and a complex lesion is an occlusion.</cite>

The industry's own summary of the documentation impact is worth quoting because it is effectively a product requirement: (cite index="89-1">Now, to accurately code procedures, health systems must precisely identify and record the arteries treated and the territories they occupy. Your physicians and cath lab staff will need to work closely together to ensure accurate recording of wire placement and treatment endpoints. Precision in the note is now precision in the code.</cite>

Almost no existing note template asks the surgeon to characterize the lesion as stenosis versus occlusion, because until this year it did not matter for coding. Every vascular note being written today at a hospital that has not updated its templates is potentially miscoded. That is a current, ongoing, quantifiable loss, and detecting it requires nothing more than a text search for deleted code numbers.

Artifact 5 — Bundling, Conflict, and Modifier Evidence Rules

The job. Encode the pairwise relationships between codes — which cannot be reported together, which include which, which require a modifier to coexist — and, for each modifier, the specific assertion the note must contain to support it.

Why it must exist separately. These rules are cross-cutting. They do not belong to any single procedure profile, because the same rule applies across many procedures. Control of bleeding is bundled into every operation. Wound closure is bundled into every operation. Diagnostic endoscopy is bundled into every surgical endoscopy. Modeling these inside procedure profiles means duplicating them dozens of times and updating them dozens of times.

Where the content comes from. The National Correct Coding Initiative publishes two things. The edit tables are machine-readable pairs of codes that cannot be billed together, with an indicator saying whether a modifier can override the edit. Those you ingest

directly; there is no value in transcribing them. The Policy Manual is prose, and it is the more valuable of the two, because it contains the *reasoning* — and the reasoning is what tells you what the note must say.

Four principles govern nearly everything, and they are stated in almost identical language across chapters: report to the greatest specificity possible; report a code only if every service it describes was performed; never report multiple codes where one code covers the work, because (cite index="57-1">This type of unbundling is incorrect coding</cite>; and finally, the one that catches the most people — (cite index="57-1">HCPCS/CPT codes include all services usually performed as part of the procedure as a standard of medical/surgical practice. A provider/supplier shall not separately report these services simply because HCPCS/CPT codes exist for them.</cite>

The dual-purpose rules are the ones that prove your product understands surgery. Some rules require the note to document something *and* forbid billing for it. The clearest example is iatrogenic injury — an injury caused by the surgery itself. Clinically, the note absolutely must record it; it is a complication, it is required by federal regulation, and it is essential for the patient's subsequent care. For billing, the repair is explicitly not separately reportable: (cite index="68-1">If an iatrogenic laceration/perforation occurs during the course of another procedure, repair of the laceration/perforation is not separately reportable. Treatment of an iatrogenic complication of surgery such as an intestinal laceration/perforation is not a separately reportable service.</cite>

A system that says both things at once — document this, do not bill it — reads as a clinical colleague rather than a billing enforcement mechanism. A system that only says "do not bill this" reads as adversarial. The difference is the entire adoption question, and it is determined by how you write the rule text.

Modifier evidence rules are the highest-yield, lowest-ambiguity content in the entire knowledge base. Each modifier has a specific sentence that must exist. For co-surgery, meaning two surgeons of different specialties each performing a distinct portion of one procedure, (cite index="107-1">Each surgeon must document separate operative notes for the specific procedure, document the co-surgeon's name, the reason the co-surgeon was needed (medical necessity), and reference the work performed by the co-surgeon in the body of the operative report.</cite> That is four checkable assertions from one sentence, and three of them are within a single note.

Interestingly, the interoperability standard already encodes the same expectation from the structural side: (cite index="88-1">There are occasionally co-surgeons. Usually they will be billing separately and will each dictate their own notes... Typically two Operative Notes are generated; however, each will list the other as a co-surgeon. Any assistants are identified as a secondary performer (SPRF) in a second performer participant.</cite> Two independent authorities describing the same requirement is a strong signal that the rule is stable and worth building.

One gating dependency you must resolve. Every modifier check should be conditioned on the Medicare Physician Fee Schedule indicators for the specific code. Those indicators tell you whether co-surgeons are permitted at all for that procedure, whether an assistant is payable, whether the code is bilateral-eligible, and what the global period is. This file is public, free, and updated quarterly. Without it your modifier engine is inferring things it could simply look up. Ingest it.

Artifact 6 — Boilerplate and Redundancy Library

The job. For each procedure, identify which text is invariant — the same in essentially every instance of that operation — and which is case-specific and must be captured from the surgeon. Provide the invariant text pre-composed, keyed to insertion points.

Why this is the artifact nobody else is building, and why it may matter most. You raised this yourself and framed it as a redundancy question: why should the surgeon repeat himself. It is a better idea than you gave it credit for, and I want to make the case.

Every artifact so far *adds* work. Each one finds something missing and asks the surgeon to supply it. Even if each request is individually justified, the cumulative experience is of software that generates tasks. This artifact does the opposite: it removes work. It is the only one that makes the surgeon's life unambiguously easier, and it is therefore the one that buys you permission to do everything else.

Consider what is actually invariant in a laparoscopic cholecystectomy. Patient identification and consent verification. Timeout performed. Positioning. Anesthesia induction and airway. Preparation and draping. Access technique and pneumoperitoneum establishment. Standard port placement. Standard closure sequence. Sponge, needle, and instrument counts. Patient extubated and transported to recovery in stable condition. None of that varies meaningfully between cases, and the surgeon dictates all of it, every time, from memory.

What is genuinely case-specific is much smaller. The findings. The degree of inflammation and adhesion. Whether the critical view of safety was achieved. Any anatomic variant. Whether a cholangiogram was performed and what it showed. Whether conversion occurred and why. Complications. Estimated blood loss. Specimen disposition.

The invariant portion is the majority of the words and close to none of the information. Pre-composing it and asking the surgeon only for the case-specific remainder is a large reduction in effort that costs nothing in accuracy.

Two constraints, and they are firm.

First, this must not become a template that appears on screen and demands filling. You stated this constraint clearly and you are right to. The interaction should be that the surgeon dictates or types naturally, and the boilerplate composes into the note behind that, at defined insertion points, visible and editable but never blocking. The surgeon should experience it as the note already being partly written, not as a form.

Second — and this is a genuine risk that will be raised by any compliance officer worth their salary — pre-populated text that describes things that did not happen is a serious problem. If the library asserts "sponge and needle counts were correct" and they were not, the note is false, and a false operative note is a legal exposure far larger than any coding error. The safeguard is a strict rule about what qualifies as boilerplate: **only text that is either verifiable from another system of record or that the surgeon must affirmatively confirm.** Counts come from the circulating nurse's intraoperative record and can be pulled from it rather than asserted. Timeout completion comes from the same source. Positioning and draping are low-risk and genuinely invariant. Findings, complications, and anything about the patient's condition are never boilerplate under any circumstances.

Get that boundary right and this artifact is your most valuable one. Get it wrong and it is a liability. It deserves careful design rather than a quick pass, and I would want to write the inclusion criteria explicitly rather than leaving them to judgment.

Part Four: Hospital variation and the override model

You raised a concern that I want to address directly, because I think it contains a hidden assumption that will cause trouble if it goes unexamined. You said that legal requirements in an operative note are "kind of subjective based on the hospital and the user," and that you do not want to lock hospitals into doing things a certain way.

The second half is correct and important. The first half is not, and the distinction matters.

Some requirements are genuinely fixed and are not subject to hospital preference. The federal Conditions of Participation are not configurable. A hospital that participates in Medicare must produce an operative report containing a specific set of elements, and no amount of local policy changes that. The interpretive guidance enumerates them: (cite index="19-1">name and hospital identification number of the patient; date and times of the surgery; name(s) of the surgeon(s) and assistants or other practitioners who performed surgical tasks (even when performing those tasks under supervision); pre-operative and post-operative diagnosis; name of the specific surgical procedure(s) performed; type of anesthesia administered; complications, if any; a description of techniques, findings, and tissues removed or altered</cite>, plus the description of tasks performed by others and any implanted devices.

If your product permits a hospital to switch off the requirement for the anesthesia type, you have built a tool that helps a hospital fail a survey. That is not flexibility; it is a defect.

What genuinely varies is real and substantial, but it is a different set of things. Which accrediting body the hospital uses, and therefore which standard citations apply and how they are numbered. The permitted time window for note completion, which the Joint Commission explicitly delegates: (cite index="26-1">An organization's policy, based on state law, would define the timeframe for dictation and placement in the medical record.</cite> Whether counts are documented by the surgeon or the circulator. Whether wound classification is a surgeon or nursing responsibility. Whether implant details live in the note or a separate implant log. The exact wording of teaching-physician attestations. Section ordering, terminology, house style. Whether the hospital participates in the Commission on Cancer, which triggers a substantial set of additional mandatory synoptic requirements. Whether it participates in the Society of Thoracic Surgeons database. And of course its own coding conventions, bundling preferences, and payer-specific rules, which you already have prompted out for several subspecialties.

Three tiers, therefore, not two. I would model overrides as follows, and the middle tier is the one that does the real work.

Tier one is **fixed**. Federal regulatory elements. Not configurable. If a hospital asks, you explain why and cite the regulation. This is a feature: it is what allows you to tell a compliance officer that your system cannot be configured into non-compliance, which is a sentence they will remember.

Tier two is **suppressible with consequence**. Code-selecting facts and modifier evidence. A hospital may decide it does not want a particular prompt — perhaps their surgeons find it disruptive, perhaps their coders handle it another way. That is a legitimate choice. But the suppression should be logged, attributed, and accompanied by a plain statement of what it costs. "Suppressing the hernia defect measurement prompt. Estimated exposure based on your volume: X cases per year at the lower code." Not a block, not a nag. A recorded decision with a number attached. Chief financial officers respond very differently to a suppression they signed off on with a dollar figure in front of them.

Tier three is **freely overridable**. Terminology, section order, boilerplate wording, house style, additional local requirements. Silent, no logging, no friction. This is where your custom-rule engine should feel effortless.

On conflicts. You said that where a user rule conflicts with a suggestion, the user rule should override. I agree within tiers two and three, with one addition: the conflict should be surfaced rather than silently resolved. If a hospital rule says to always bill a particular pair of codes together and the National Correct Coding Initiative edit table says that pair is mutually exclusive, honoring the hospital rule silently makes your system complicit in a pattern that will eventually be audited. Honoring it *while displaying* the conflict and its source keeps the hospital in control and keeps you clearly documented as having flagged it. That is a materially different position to be in if the question is ever asked in a legal or regulatory setting, and it costs you nothing.

Part Five: Competitive positioning, revisited

Your Blue Ocean narrative is correct in structure and I would not change its shape. Front-end ambient documentation companies write down what they hear and have no coding intelligence. Back-end autonomous coding companies read what was written and cannot recover what was not. You sit in the gap. That framing is sound.

I want to sharpen one element of it, because the phrasing you are currently using is slightly weaker than the argument deserves.

You currently say that if a surgeon writes a bad note, the best back-end coder still cannot bill the missing parts. True, but abstract. Here is the concrete version, and it is more damaging to the competitive position you are attacking:

A surgeon writes "repaired a ventral hernia, approximately 3 cm defect, mesh placed." That note is clinically adequate. It is legally compliant — it names the procedure, describes the technique, identifies the implant. It will pass every completeness check the hospital currently runs. An autonomous coding engine will read it and confidently assign a code. It will assign 49591, and it will be *correct to do so*, because that is the only defensible code given the word "approximately." No error will be flagged anywhere in the system. The claim will be clean. It will be paid on first pass. It will contribute to a ninety-five percent clean claim rate on somebody's dashboard.

And two hundred thirty-three dollars will be gone, permanently, invisibly, on that case and on every similar case, forever.

That is the argument. The back-end tools are not failing at their job. They are succeeding at their job, and their job is insufficient, because the information was destroyed before they received it. There is no possible improvement in downstream coding accuracy that recovers it. The only intervention that works is one that occurs while the surgeon is still in a position to know the answer.

That framing is stronger than "garbage in, garbage out" because it does not require you to claim the competitors are bad. It concedes they are good and shows that their category has a ceiling. Ceilings are more persuasive than flaws, particularly to investors who have already met the competitors and found them impressive.

I would also note one thing about your competitive analysis that you may want to verify independently: the specific funding and customer claims about individual competitors in your document should be re-checked against current sources before they go into an investor deck. Competitive claims that turn out to be stale are disproportionately damaging, because the investor you are pitching may well know those companies better than you do.

Part Six: What I need from you

I want to be direct about the dependencies, because three of these gate real work and guessing at them will produce artifacts that look right and fail in production.

First and most important: does the case posting reach your system? As discussed in section 1.3, this determines whether the detection artifact is a small disambiguation layer or a large classification system. It is the difference between two quite different pieces of work and I do not want to build the wrong one.

Second: do you have the American Medical Association CPT license? CPT code descriptors are copyrighted. You cannot ship them, expose them through an interface, or embed them in a knowledge base without a license, and the license terms constrain redistribution in ways that affect architecture. Every decision tree in Artifact 4 depends on descriptor text. This is a procurement gating item, not a detail, and it should be moving in parallel with the technical work rather than after it.

Third: can you obtain de-identified operative notes from Vandalia? Even a few hundred, even from a single service. This determines the quality of the element dictionary's surface-form map more than any other input. I can write a plausible list of how surgeons phrase things. A plausible list is a list of how I imagine surgeons write. The gap between those two is exactly where a completeness engine generates false positives, and false positives are what get software switched off.

Fourth, and lower stakes but worth knowing: the subspecialty coding rules you have already prompted out. If those exist, they are the seed of Artifact 5's hospital-specific layer and I should build the schema to accommodate their shape rather than inventing a shape they will not fit.

Part Seven: Where I would start, and why

If I had to pick a single place to begin that produces value before any clinical reasoning is involved, it would be this.

Scan Vandalia's operative note template library for deleted CPT codes.

Cerner's PowerNote templating mechanism creates conditions that make this problem near-certain. Templates are saved as "precompleted notes" and shared across units. The governance is informal by design: (cite index="74-1">Precompleted Notes can be shared. Units may wish to create shared Operation Report templates for commonly performed procedures in this way... Please be aware, a shared note can be edited or deleted by any user. We recommend keeping a backup.</cite> And the vendor documentation itself warns that (cite index="74-1">Please be careful when managing PowerNote templates. It is NOT possible to recover accidentally deleted templates.</cite>

Templates created by individual units, editable by anyone, with no version control and no recovery, are templates that nobody updates when the American Medical Association deletes sixteen vascular codes in January. The exposure is almost certainly live and almost certainly nobody has looked.

The scan requires no clinical artificial intelligence, no CPT license for the descriptors, no model at all. It is a text search for a list of deleted code numbers. The output is a document that says something like: *thirty-seven of your two hundred fourteen shared operative note templates reference CPT codes that no longer exist, concentrated in vascular and general surgery, affecting an estimated N cases per month.*

That artifact does several things at once. It demonstrates that you understand their environment specifically rather than generically. It produces a finding they can verify themselves in minutes. It creates urgency without any projection or estimate. It gives you a reason to be handed the template library, which is the input you need for everything else. And it establishes the relationship on the footing of *we found something real* rather than *here is what we could do for you*.

After that, the build order I would follow is: the universal element checks, because they apply to every note regardless of specialty and are the least ambiguous content in the system; the affirmative-negative cluster, meaning estimated blood loss, specimens, implants, complications, and drains, because these are where completeness failures actually concentrate; the modifier evidence rules, because they are high-value and unambiguous; then the vascular lower-extremity work, because it is the most time-sensitive;

then hernia, because it is the best demonstration case; then coronary bypass, because it is the most complex and highest value per case.

I have deliberately not attached durations to any of that. The sequence is what matters, and the sequence is driven by dependency and by risk, not by effort.

Closing

The core of what you are building is sound, the evidence supports it more strongly than your current materials claim, and the architectural instinct — intervene at authoring rather than at coding — is the correct one and is in fact the conclusion the systematic review literature reached independently.

The three things I would change are these. Reorganize the knowledge base around code-selecting facts rather than around codes, because facts are checkable and codes are conclusions. Replace the contestable figures in the return-on-investment model with the verifiable ones, because a single case with real arithmetic is worth more to a hospital finance executive than a four-digit percentage. And build the boilerplate library early rather than late, because it is the only artifact that reduces a surgeon's work rather than adding to it, and it is what buys you the standing to do everything else.

Tell me about the case posting and I will start on detection. Or tell me which artifact you want first and I will start there instead.